

Automotive Skid Elevator Station

PLC software for a body-shop skid elevator connecting roller bed conveyors across three vertical levels — built as a hands-on exercise in motion control and PLC-to-drive communication.

Author

A. Fernando Gola

Siemens S7-1511, TIA Portal V17

Project status: in active development

2026

Introduction

This project is inspired by a skid elevator and roller bed transfer point similar to those found on an automotive body-shop line where I currently work in electrical maintenance. It's not a copy of any real installation — everything here, code and data structures alike, was built from scratch as a learning exercise, using only the general mechanical behavior as a reference.

The real goal is motion control: understanding how a PLC actually talks to a drive, not just making a sequence run. So there's no real drive anywhere in this project — both axes (E156's lift and RB156's braking conveyor) are simulated through a struct that mimics a real drive's interface: Control Word, Status Word, velocity setpoint/feedback, a fault code, and — only for E156 — a position setpoint/feedback, since it's the only one that closes a position loop.

Both simulated drives are modeled after SEW hardware I've seen on the plant floor, deliberately not Siemens — using a Sinamics drive would have let TIA Portal configure the whole telegram, and its addresses, automatically. Doing it by hand for a vendor TIA doesn't know forced me to actually think about what's inside that telegram.

In a real plant this kind of station rarely runs on a single PLC — the roller beds just up and downstream usually belong to a different section, sometimes a different controller. This project mirrors that at a smaller scale: RB154 (upstream) and RB159/RB160 (downstream) are modeled only as an external, simulated interface, not full stations. The platform is a standard S7-1511 (not "T", not "F") — motion is handled entirely by the simulated drives, not a Technology Object, and safety is handled entirely by a Pilz relay, not PROFIsafe. Built in TIA Portal V17, with LAD for interlocks and alarms and SCL for state machines and structured data.

System Overview

The station moves a car body on a skid vertically between three levels, using four roller beds and one elevator.

- RB155 — receives the skid from upstream and feeds it into RB156.
- RB156 — the last roller bed before the elevator, physically mounted on top of the E156 platform (they move together). Its own SEW drive brakes the skid smoothly on arrival — open-loop, velocity only, no position feedback.
- E156 — the elevator itself. A second, separate SEW drive with absolute-encoder feedback closes a real position loop, raising or lowering the platform — and RB156 riding on it — between levels.
- RB157 — receives the skid at the high level.
- RB158 — receives the skid at the middle level.

RB159/RB160 (downstream of RB157/RB158) and RB154 (upstream of RB155) are outside this station's scope — modeled only as external status sources, described in Software Architecture.

Mechanical Layout

E156 isn't a scissor lift — it's a platform-and-chain elevator: a motor mounted at the top drives a chain that raises and lowers the platform, guided by two vertical columns on either side. RB156 rides directly on that platform.

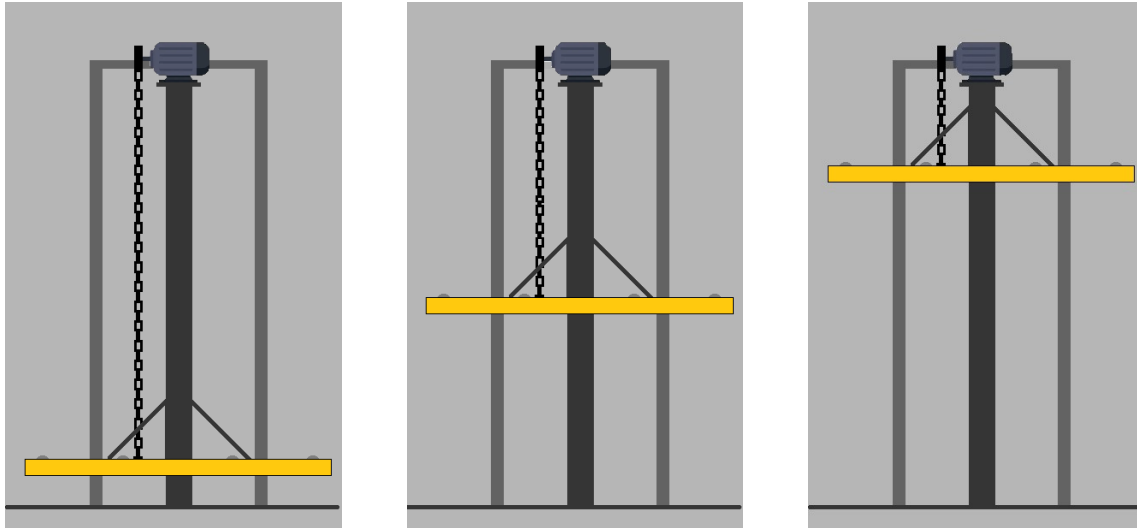


Fig. 1 — E156 at its three positions: low, middle, high.

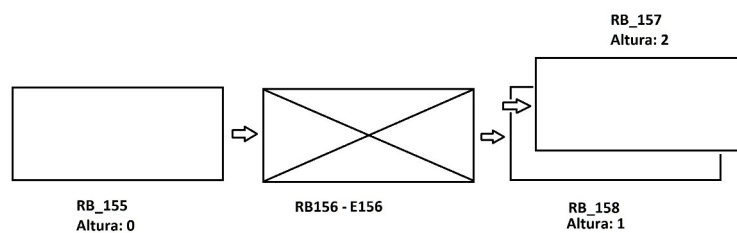


Fig. 2 — Station layout: RB155 feeds the combined RB156–E156 assembly, which discharges to RB157 (high) or RB158 (middle).

- Low — level with RB155, to receive a skid.
- Middle — level with RB158.
- High — level with RB157.

Vertical movement is handled by E156's simulated closed-loop axis — the PLC only sees position/velocity setpoints and feedback, never a raw encoder count; the loop itself is internal to the simulated drive.

Each level has its own proximity sensor, independent of the encoder, used to cross-check the reported position (see Control Sequence). Two more sensors (ExtraHigh/ExtraLow) sit past the normal travel limits as a last-resort overtravel safeguard.

The platform also carries a set of mechanical blocks — pins that extend from the platform once it's parked at a level, locking it against the frame so it physically can't descend. They're a mechanical backup against a drive or brake failure, not something related to holding the skid in place; they retract before any move and re-engage once the platform settles at the new level.

Control Sequence

Transfer handshake

Each roller bed exposes a small set of status bits (Empty / Occupied / Transfer / Fault) that its downstream neighbor checks before requesting a transfer, plus a pair of commands (SetToNext / ResetToNext) to formally request and acknowledge it. Same pattern locally and, through the simulated external interface, with RB154 / RB159 / RB160.

RB156 — open-loop drive

RB156's own SEW drive is open-loop: it takes a velocity setpoint and runs its own internal ramp to bring the skid to a smooth stop, with no position setpoint or feedback anywhere. Its simulated telegram: Control Word, Status Word, SpeedReference/ActualSpeed, and a fault code — the same shape as E156's, just without the position fields.

E156 — closed-loop axis

E156 is the one axis that actually closes a position loop — it has to stop precisely at one of three levels, not just ramp to a stop. Its telegram adds SetPosition/ActualPosition on top of the same Control Word / Status Word / velocity / fault-code shape.

The PLC never assumes the axis is ready just because it sent a command — each step of the enable sequence (release stops, ON, Enable Operation) only advances once the Status Word confirms it. An early version skipped these checks; it worked fine until the axis took a bit longer than usual to respond, then failed silently.

Two move types share the same state machine: a manual/jog move (pure velocity, used for reference recovery and manual operation) and an automatic positioning move (target position plus target velocity). The ramp shape itself lives inside the simulated drive, not recalculated by the PLC on every move.

Position validation instead of classic homing

Since every level has its own sensor, there's no classic homing routine run once at power-up. Every arrival is checked live instead: ActualPosition has to land inside a tolerance band around the setpoint (IN_RANGE) and the matching level sensor has to be active at the same time. If they disagree, the encoder-to-mechanics correspondence is no longer trusted — drift, a coupling issue, an encoder swap — and the axis faults instead of trusting a number that might be wrong.

Recovery doesn't move the axis to a computed position — that's exactly what's in question. The elevator gets jogged manually onto a known level sensor, then a "set reference point" command re-anchors the encoder count to that known position. The encoder itself is never touched, only its correspondence to reality.

Manual jog behavior

In manual, a direction command doesn't run indefinitely — it stops at the nearest level in that direction, even with the button held down. Reversing direction sends it to the next level the other way, so an operator can't drive past a valid stop.

Software Architecture

Component blocks

Every physical component gets its own FB and its own instance DB — FB_RB155, FB_RB156, FB_E156, FB_RB157, FB_RB158 — instead of one reusable multi-instance FB shared across the four roller beds. It's less "correct" by the textbook, but during a line stop, finding a station's logic by name beats saving four similar-looking blocks in the project tree.

I/O abstraction layer

FC_I/O_Interface is the only block that touches physical addresses — it copies raw I/O into DB_Global on the way in, and DB_Global's actuator commands out to physical outputs on the way out. Everything else only ever touches DB_Global.

UDT_RB, and why RB156 and E156 don't use it

RB155, RB157, and RB158 are identical, so they share a PLC data type (UDT_RB: ACT, SNS, CMD, ALM, STS) instantiated once per station inside DB_Global. RB156 and E156 don't use it — both carry their own DRIVE struct (Control Word, Status Word, SpeedReference/ActualSpeed, a fault code, and, for E156 only, SetPosition/ActualPosition) that the other three don't need. Forcing them into the shared type would leave every other instance carrying dead fields, so each gets its own plain Struct instead, with the same ACT/SNS/CMD/ALM/STS layout plus its own DRIVE on top.

RB156 and E156 are physically the same assembly — RB156 rides on E156's platform — but stay two separate blocks in software, since each owns its own drive and its own logic. Physical stacking isn't what decides data ownership here; each component just owns whatever hardware is actually its own.

No Input/Output/InOut parameters are used to pass component data around — every FB reads and writes DB_Global directly, by its full symbolic path. With DB_Global organized the way it is, that direct access reads fine, and didn't seem worth adding a parameter layer for. Each FB's own Static only holds what's local to it: timers, edge memories, state-machine variables.

HMI interface layer

FC_InterfaceConnection_HMI does the same job toward the HMI side: it copies what the HMI actually needs into a dedicated DB_HMI, and copies commands back the other way. The panel only ever touches DB_HMI, so internal data changes don't ripple into every screen.

External interface

DB_External_Interface simulates the link to RB154 / RB159 / RB160, exposing only Occupied / Faulted / ReadyToTransfer per station — plus one toggling heartbeat bit per link (not per station), checked through an edge-triggered watchdog timer, so a dead link is caught within a few seconds instead of just reading stale data as current.

I/O addressing

Digital I/O and the drives' word-level telegram data (Control/Status Word, velocity, position, fault code) share the same byte-addressed process image, so every new word had to be placed by hand to avoid overlapping a digital module's range — something a real drive integration would handle automatically. Tags are split into one table per station instead of one shared default table.

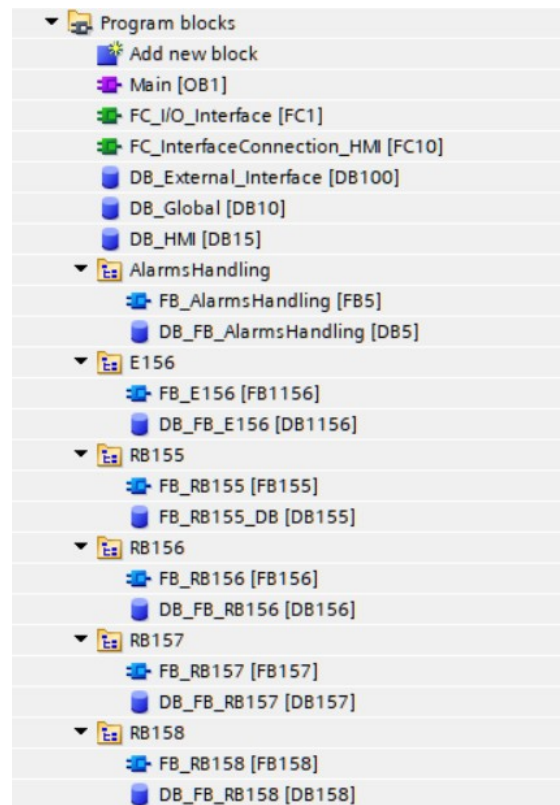


Fig. 3 — Program blocks, as implemented in TIA Portal.

Alarms Management

Distributed detection

Each component's own FB detects its own faults — thermal protection, timeouts, position mismatches, overtravel, drive fault codes — and writes straight into its own ALM struct in DB_Global. FB_AlarmsHandling only reads and consolidates that into one HMI-facing summary; it doesn't evaluate anything itself.

Latching, not re-evaluation

Alarm bits use Set, not a plain coil, so a fault that clears itself in a few scans — a brief overcurrent — stays visible until someone acknowledges it. Same for the general Fault bit: only an explicit Reset from the HMI clears it, never an automatic re-check. An early version used a plain coil for one alarm, which self-cleared the instant the Fault bit it triggered went true in the same scan — invisible, even though the real condition was still there.

Alarm interlock

Rather than track a "first cause" code, each alarm's Set is gated by an interlock: it only fires if no fault is active yet. Once the first real alarm sets Fault, every other alarm-setting rung downstream is locked

out — so whatever tripped first is the one that stays visible, without needing a separate variable to capture it. A few alarms — thermal protection, breaker trips — skip the interlock and always show, since they matter even while everything else is already frozen.

Everything defaults to non-retentive, alarms included — a fault re-detects itself within a scan of any restart, and TIA only lets Retain be set for a whole UDT instance at once anyway, not just ALM on its own.

Safety Integration

Doors, light barriers, and e-stops wire exclusively to a dedicated Pilz safety relay, never to the PLC directly — and since the S7-1511 isn't an F-CPU, it couldn't join a PROFIsafe loop even if it wanted to. Safety decisions and motor disconnection happen entirely in hardware, independent of the PLC program.

The PLC only reads non-safety status bits from the Pilz system (SafetyOk / EmergencyOn, plus individual door/barrier status) for HMI display and to block automatic mode — never to make a safety decision itself.

Manual Mode and Manual Maintenance Mode

One Auto/Manual selector covers the whole station, not one per component — the cell is small enough that finer granularity wasn't needed. In manual, moves are still gated by the current sequence state, so an operator can't easily put the machine somewhere unsafe. A separate, key-switched maintenance mode bypasses those state checks for qualified personnel, but never bypasses the safety system itself. The SetToNext / ResetToNext override mentioned earlier is meant for exactly this kind of recovery, and only works in manual, with its own checks.

HMI

Built on a TP1200 Comfort panel, over PROFINET. Screens: Home, Alarms, and one per component — RB155, RB156_E156 (combined, since they're one physical unit), RB157, RB158. Everything goes through DB_HMI, never DB_Global directly.

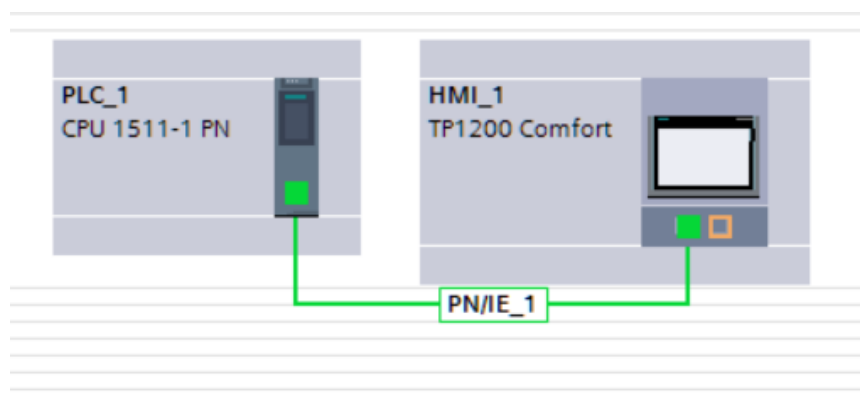


Fig. 4 — PLC-HMI network configuration.

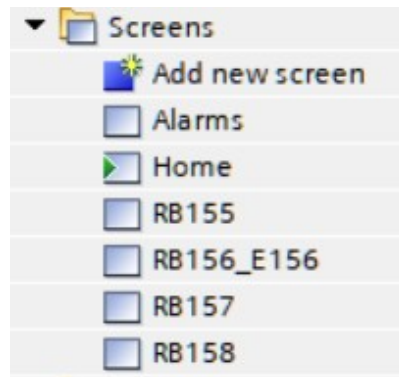


Fig. 5 — HMI screen structure.

Design Notes and Lessons Learned

This is a self-directed learning project, and the mistakes are as much the point as the result.

E156 was originally a UDT, on the general “good practice” assumption. Since its structure kept changing as I understood the process better, every new field meant editing a shared type and recompiling everywhere it was used. Restarting it as a plain Struct fixed that instantly — and RB156 later ran into the same problem for the same reason, once it needed its own drive data too.

Same trade-off with giving every roller bed its own FB instead of one reusable multi-instance block — I picked the option that’s easier to find under pressure over the one that’s more “correct.”

Working through both drives clarified what a PLC actually owns in a closed loop: request a destination and a speed, confirm the handshake, and independently check — with a plain sensor — that the reported position matches reality. RB156’s drive never needed any of that, since it’s open-loop. The two components sitting on top of each other taught a smaller, related lesson: my first instinct was to make one struct “own” the drive data for both, based on physical proximity. Wrong call — each owns its own hardware, shaped to match, regardless of who’s sitting on whom.

Simulating SEW drives instead of Siemens ones was deliberate — TIA’s own tooling would have built the telegram, and its addresses, for me. Doing it by hand meant actually reasoning through both, including the mistakes that come with it: addresses that overlapped digital I/O until I accounted for exactly how many bytes those modules already claimed.

The alarm logic had its own bug: a plain coil on an alarm tied to a condition that set Fault in the same scan would self-clear the next cycle, invisibly. Fixing it, and later replacing a planned “first cause” variable with a simple interlock instead, both came down to the same idea — sometimes the simpler mechanism, a gate instead of a stored value, is also the more correct one.

If there’s one thread through all of this, it’s that no architecture — UDTs or not, direct access or parameters, interlocks or logged codes — replaces the discipline of whoever maintains this afterward. Every choice here was made with that person in mind, including the future version of me who comes back to it in a few months.